



Production Information Systems

Topic : Object-oriented analysis and design

Chapter 7 : Unified Modeling Language State Chart Diagram- Class Diagram

ISE Department
Year 4/ Level 7

Instructor : Dr. Jomana Bashatah



Outline

1. Introduction
2. State-Chart Diagram
3. Class Diagram
4. Case Study

1.Introduction

- As discussed in last chapter, we will focus through this topic on a subset of UML diagrams that are useful in information system design.
- As use case diagrams, sequence diagrams and activity diagrams, the relevant subset includes state chart diagrams and class diagrams.

2. State-Chart Diagram

A ***state chart diagram*** is a dynamic model that shows **the various states** that an object can be in and how the object evolves through those states over time as a result of events that occur.

The state chart can capture behavior at various levels of the system, including overall system behavior, class behavior, and detailed behavior of specific objects.

UML state chart diagrams are a variant of a traditional model of computer behavior known as a finite state machine.

A ***finite state machine*** is composed of three elements: **states**, **transitions**, and **event conditions**.

2. State-Chart Diagram

- These elements are shown in figure 9.10, which is a high-level state chart diagram of a ticket object during the execution of events in the Ticket Sales system.

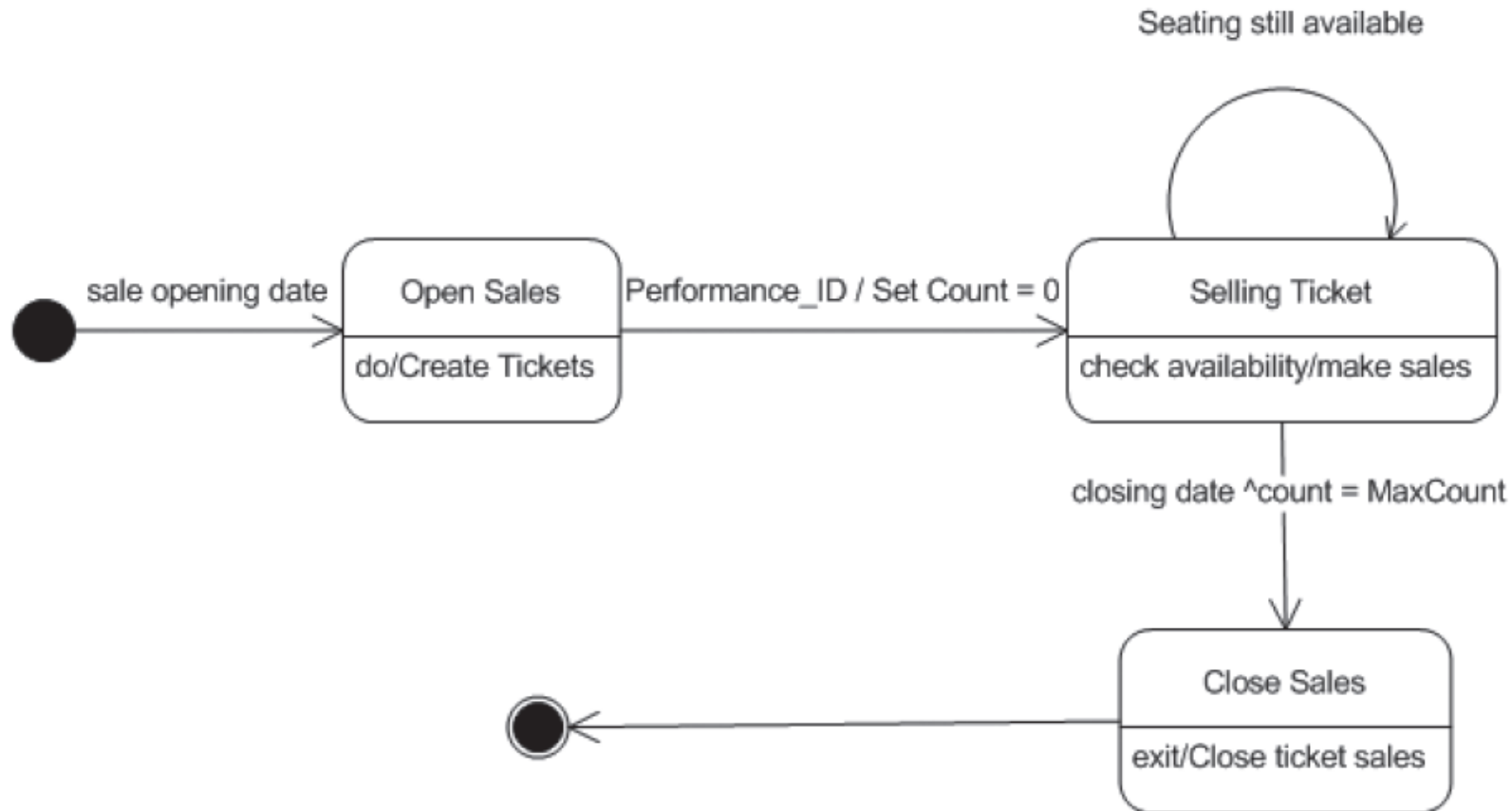


Figure 9.10 State chart diagram of instances of the ticket sales system.

2. State-Chart Diagram

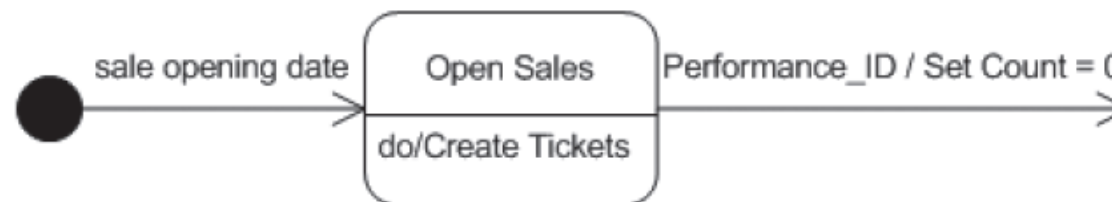
- **A state**, represented as a box with rounded corners, is a stage of an object's life cycle that has a finite duration.
- Three states of the ticket sales system are modeled.
 - In the first state, the ticket sales for a performance are being opened and tickets are being created.
 - In the last state, the ticket sales for a performance are being closed.
 - The interim state is the period during which the ticket sales are taking place.
- These three states are a fairly high level view of the behavior of an instance of a Ticket object class during its life cycle.

2. State-Chart Diagram

- **The transitions** are the connecting arcs between states. A transition is labeled with an event condition that identifies the event that triggers the transition.
 - When an object is in a particular state, the output transition of that state is **armed** (ready to be fired).
 - If a transition is armed and the event condition on an output transition is true, the transition will fire and the object's state changes from the input state of the transition to its output state.
- The event condition is labeled Event Name/[Action].
 - The **event name** is the trigger (condition) that causes the transition to fire; the optional **action statement** is an action that results from the event.
- Unlike a state, which may be occupied over a period of time, an event and the firing of its transition occur at an instant in time. Transitions can also be self-loops of a state.

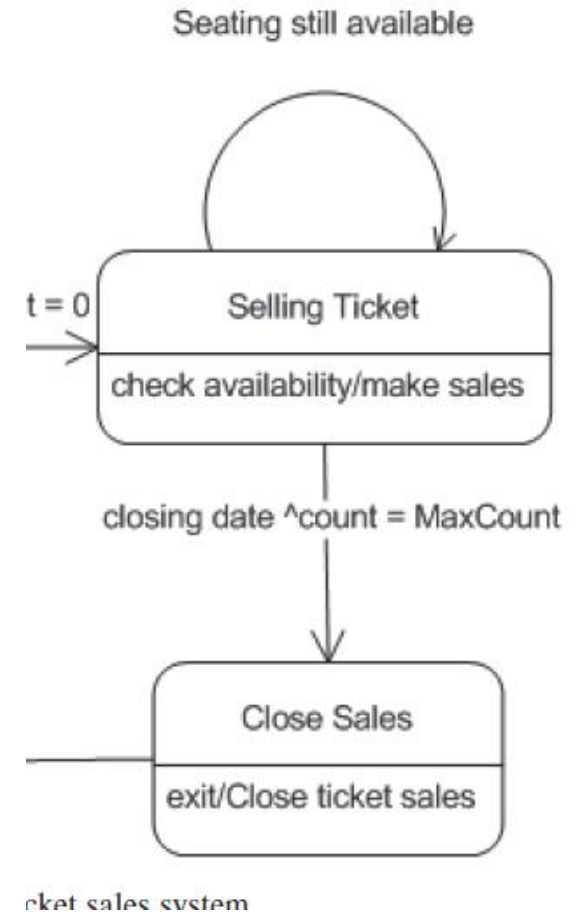
2. State-Chart Diagram

- The previous figure models the behavior of the ticket sales system. With regard to the behavior of objects of the system, there will be many such objects executing their state chart models. Let us focus just on the Ticket object class. A ticket is a claim of the owner on a seat at a specific performance.
- The initial event that begins the execution of the state chart is the opening date of the sale. Control first passes to that state in which ticket objects are being created and sales are opened.
- This activity provides an output event identifying the performance to go on sale and setting the count of total tickets sold to zero.



2. State-Chart Diagram

- The self-loop on the state “selling tickets” indicates continuing ticket sales. This continues until one or both of two events occur: the closing date *or* tickets are sold out.
- Either of these events will transition the state chart to the “close sales” state, and the state chart will terminate.
- A state chart diagram can model more details of object behavior. The UML standard has adopted Harel state charts (Harel and Politi, 1998), which specify modeling of layers of object behavior.



2. State-Chart Diagram

For example, significant object behavior is masked in the single high-level state “selling tickets.”

Next Figure exposes more details of the “selling ticket” state by showing the state machine of the ticket object.

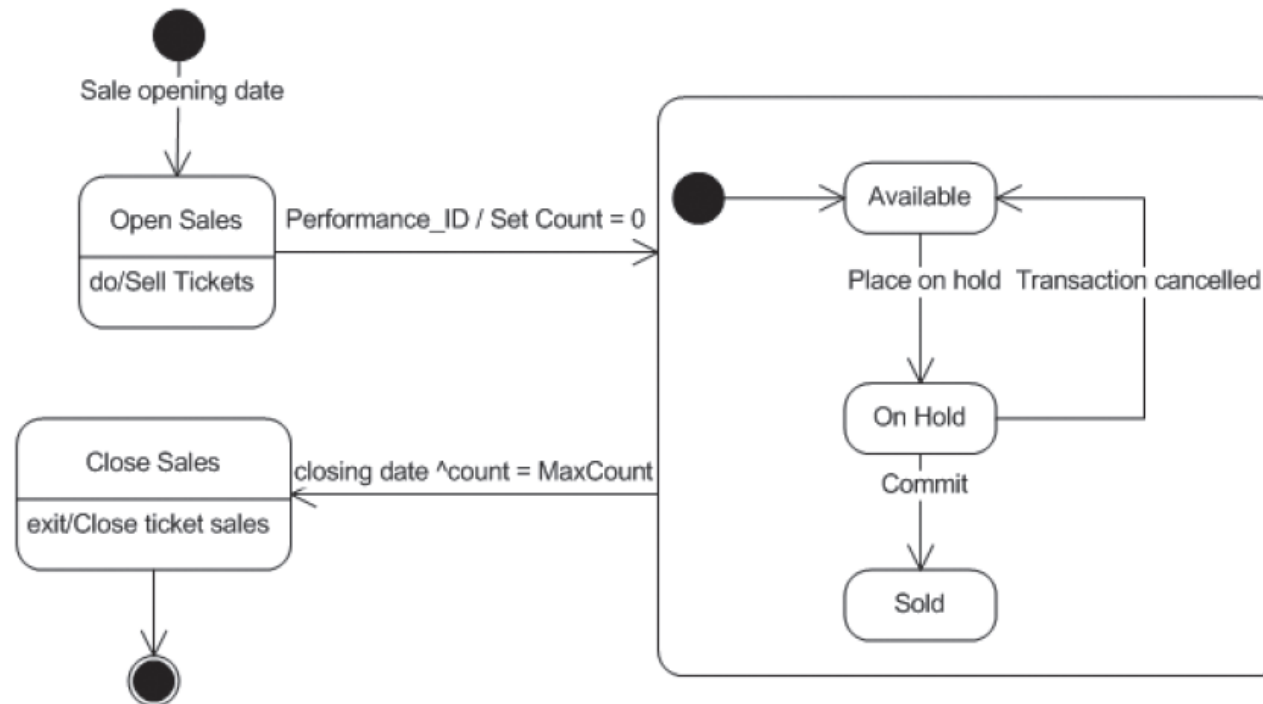


Figure 9.11 State chart diagram showing ticket object states during sales.

2. State-Chart Diagram

- Here we see that, after creation, a ticket can be in one of three states. Immediately after creation, it is in the “available” state.
- When a ticket is in the available state *and* the event “place on hold” occurs, the ticket transitions to the “on hold” state.
 - Note that the event “place on hold” is generated by the Ticket Agent object during the “make sales” use case.
- Similarly, when a ticket sale is committed, the state of the object transitions from “on hold” to “sold.” The state chart shows that during the period of sales, the states of the ticket objects are changing from “available” to “sold.”

2. State-Chart Diagram

- Presumably, when the closing date for ticket sales for a performance occurs, tickets will be in one of two states: available or sold.
- The transition labeled “transaction canceled” takes the ticket back to the “available” state.
- In the illustrated use case, sequence, activity, and state chart diagrams of the ticket sales system, we have not tried to capture the models associated with canceling a sale (returning a ticket) after the sale has been made. Implicitly, we have imposed a business rule that states “all ticket sales are final.”

3. Class Diagram

One purpose of the step-by-step construction of UML diagrams is to identify objects and to group objects into classes.

The ***class diagram*** is the most important UML model as it defines the object types that can be created by the system.

These would include software objects such as the ticket agent and database objects such as ticket entities.

After constructing use case, sequence, activity, and state chart diagrams, the analyst will have some familiarity with the requirement specifications of the case. This should suggest data elements that are required for the data model. So we will begin the discussion of class diagrams by first suggesting a data model based on relational database design rules. This will then be contrasted with the object-oriented class diagram.

3. Class Diagram

From the sequence diagram, some concepts have been identified: performance, seating, ticket, credit card, and sale.

Considering a relational database design, we can declare the structural relationship between these concepts as follows:

1. A Ticket represents a claim on a Performance and a Seat.
2. Each Performance is associated with many Tickets.
3. Each seat (which we shall call a Location) is associated with many Tickets, since the same seat is used for more than one Performance.
4. A ticket Sale is made for one or more Tickets.
5. A Credit Card is used in each sale.

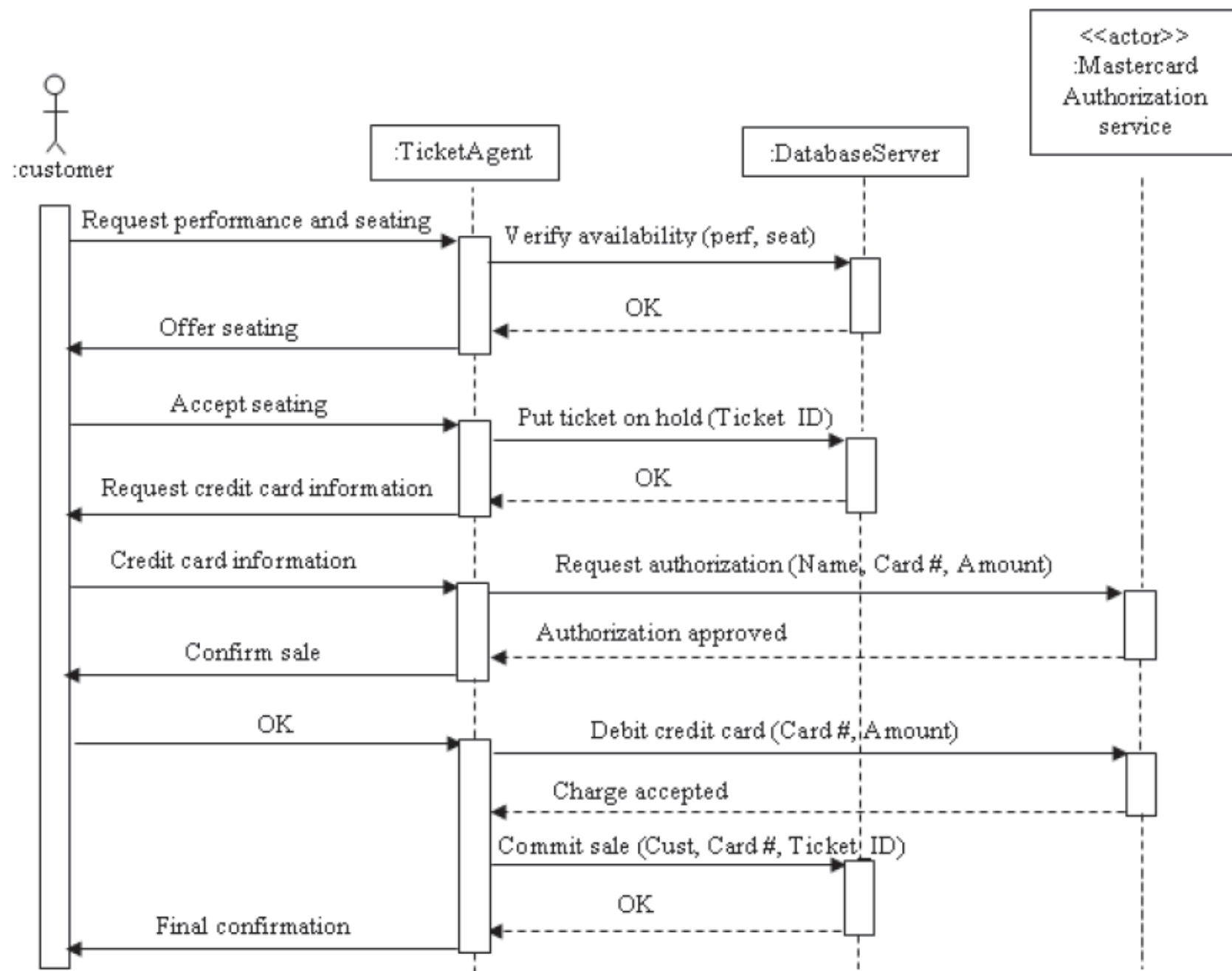


Figure 9.7 Sequence diagram with more details of scenario 1.

3. Class Diagram

- These structural relationships suggest the relational data model shown

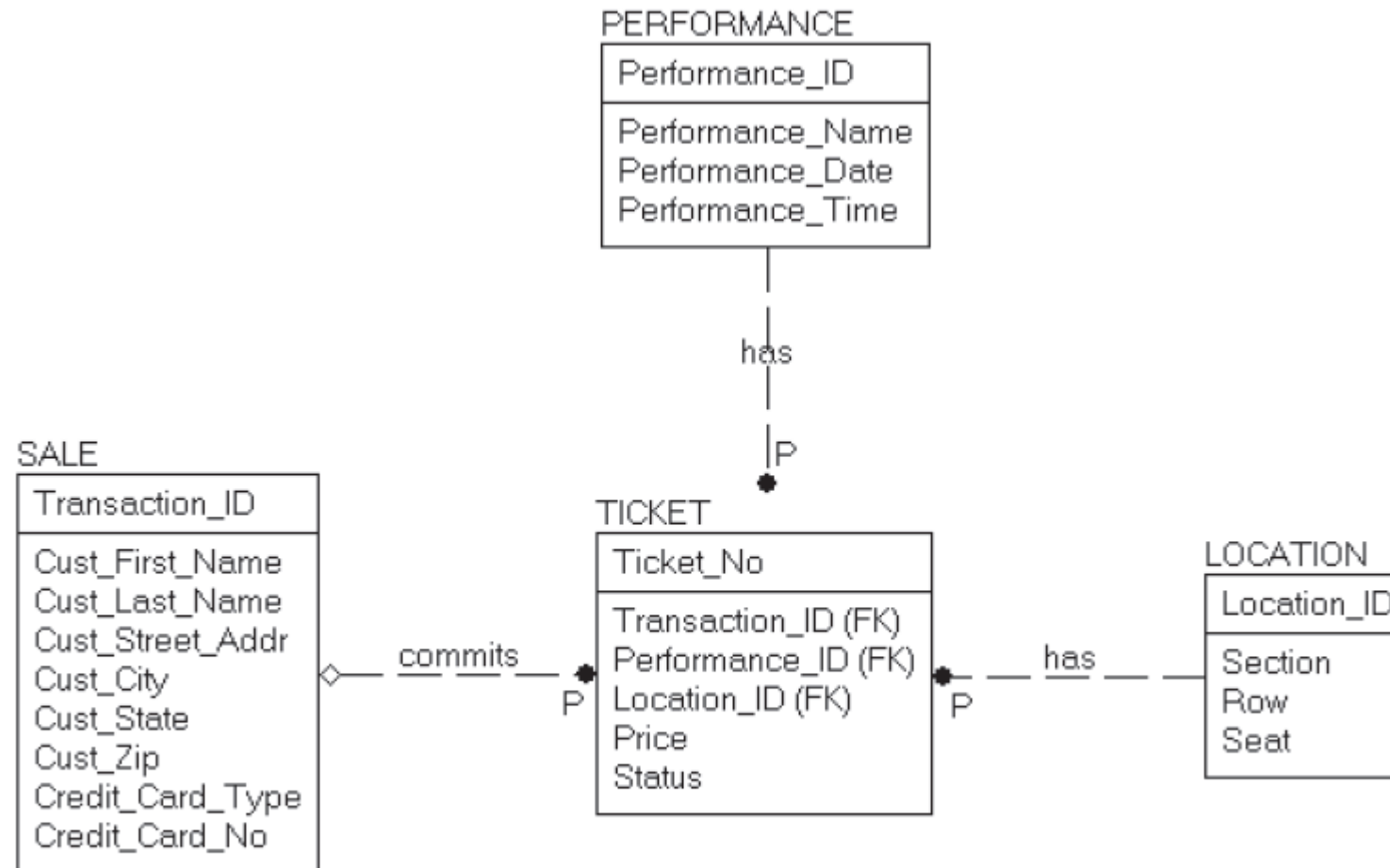


Figure 9.12 Relational model of ticket sales database.

3. Class Diagram

- Both PERFORMANCE and LOCATION have a one-to-many relationship with TICKET.
 - For a given performance, there will be many tickets, but each ticket is for one and only one performance.
 - A location has many tickets since the same location is used over and over again for different performances.
 - We have chosen to make Price an attribute of TICKET under the assumption that the same seating location for different performances may have a different price.
- The SALE entity set documents the sale information once a ticket is sold. Note that Transaction_ID is a foreign key of TICKET.

3. Class Diagram

- The same Transaction_ID can occur for more than one ticket indicating the sale of more than one ticket during the same transaction.
- Tickets not yet sold will have a NULL value for Transaction_ID in the TICKET table.
- All tickets are created on the opening day of sales, as documented in the state charts shown in state chart diagrams.
- The Status attribute of TICKET is used to keep track of whether or not a ticket is available, on hold, or sold.

3. Class Diagram

- The previous figure could be modified in several ways and there are different designs that could be considered, but for the purposes of this explanation, it is an acceptable data model of the ticket sales database as we understand it from the previous diagrams.
- This could indeed be the outcome of the analysis of data elements in the sequence diagram from the “make sale” use case if we were to implement the data requirements in a relational database.
- In that sense, UML is useful in documenting the requirements of an information system and may be preferred to other modeling techniques such as IDEF or DFD, based on the taste of the analyst.

3. Class Diagram

3.1 Components of a class

- The correspondence between the relational data model world and the OO data model world is rather casual as they each have a different philosophy, emphasis, and implementation requirements. However, it is useful to draw parallels between the two to explain the UML class diagram.
- Roughly speaking, what is referred to as an “entity set” in relational data models is analogous to a “**class**” in OO data models.
- A class diagram is composed of **classes**, which are the templates for sets of similar objects that have the same attributes and behavior.
- In a class diagram, each class is represented by a box with three distinct sections, as shown in Figure 9.13.

3. Class Diagram

3.1 Components of a class

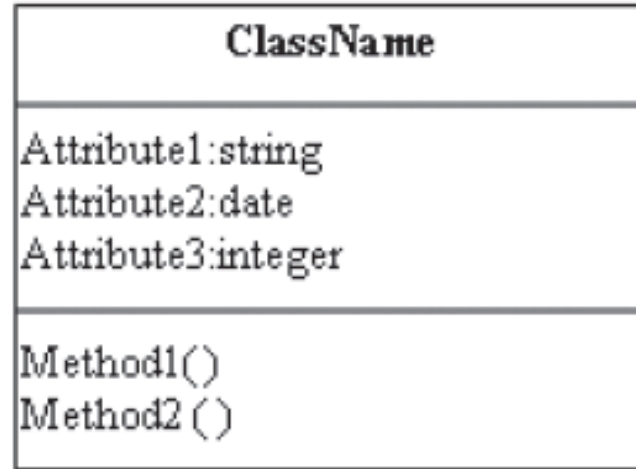


Figure 9.13 Components of a class.

3. Class Diagram

3.1 Components of a class

The name of the class is given in the top panel. This is a descriptive name and is analogous to the name of the entity set in relational data models.

The center panel contains the attributes of the class and their data types. Object-oriented models support the usual data types of string, integer, date, and so forth, as well as user-defined data types.

Attributes of the OO data model are analogous to the attribute names in relational data models.

So far a class looks much like an entity set. However, the third panel contains the behavior of the class in the form of operations that the class can perform on its data — that is, on the values of its attributes.

3. Class Diagram

3.1 Components of a class

These operations, which are functions that can be called (invoked), are sometimes referred to as “operations” and sometimes as “methods.” We will continue to use the term “methods.”

As stated in the beginning of this chapter, an object combines both data (attributes) and behavior (methods). The method can be invoked by other objects or may be invoked by the object itself.

In the relational model, data are operated on by SQL commands from the user or from a user program. Data and functions that operate on data are separate.

The operations and associated code are not built into the entity set (table of the database).

In OO data models, the operations that are performed on the data are part of the object definition and reside in the object itself.

3. Class Diagram

3.1 Components of a class

- The four entity sets in Figure 9.12 are shown as classes in Figure 9.14.
 - Note the declaration of the data type for each attribute. Primary keys are not used to uniquely identify an object because each object of a class will automatically be assigned a unique object ID (OID) by the system when the object is created.
- Methods are shown in the lower panels of the classes. The method “Commit ()” in the Ticket class, when invoked, results in the change of status of the Ticket object from “on hold” to “sold.”
 - This behavior was described in the state chart in Figure 9.11. It is invoked by the function call “Commit sale ()” that appears at the bottom of the sequence diagram of Figure 9.7.
 - The Ticket class has an attribute “Status” that is assigned the value of the state of the ticket (available, on hold, sold).

3. Class Diagram

3.1 Components of a class

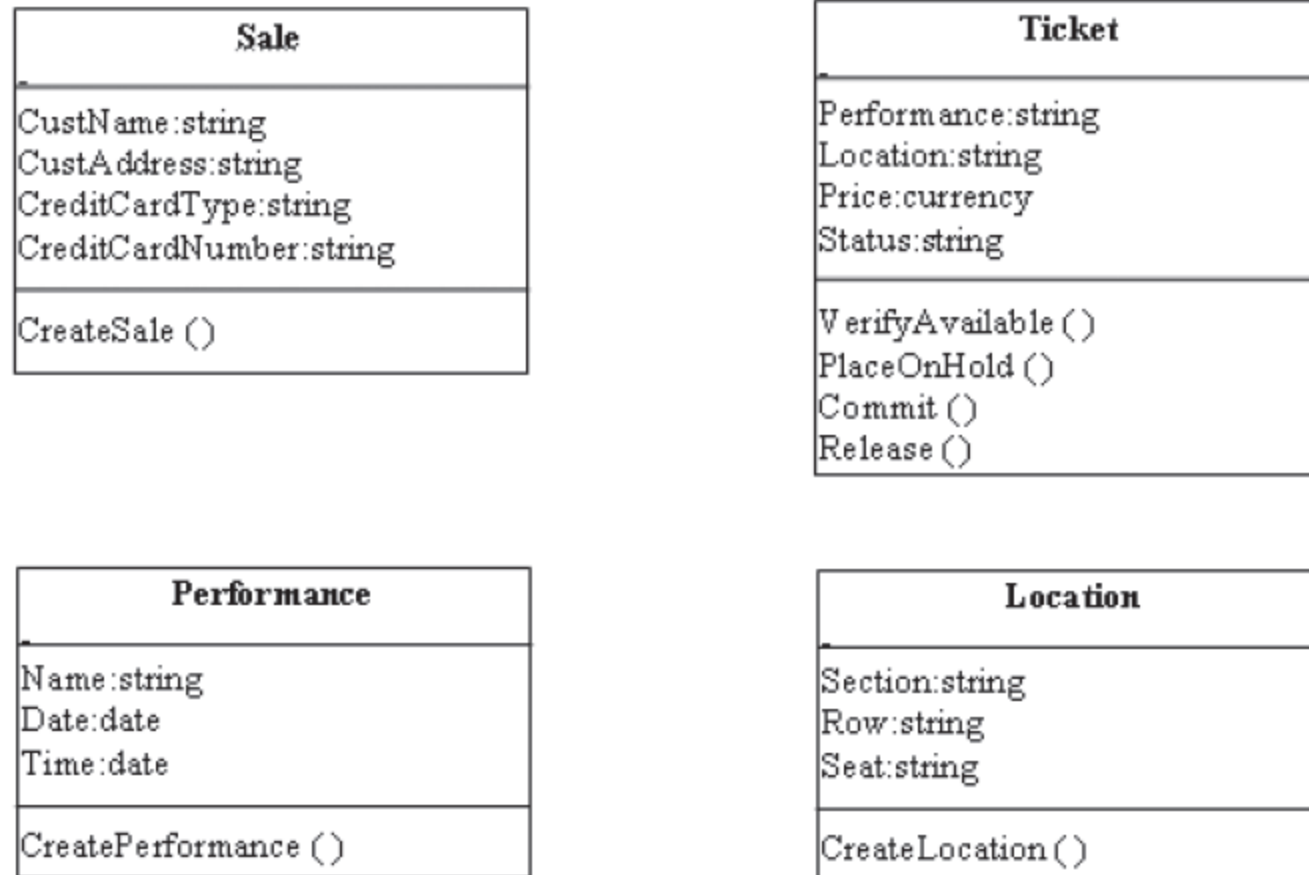


Figure 9.14 Ticket sales classes.

3. Class Diagram

3.2 Associations

- An **association** in a class diagram is analogous to a relationship in a traditional E-R or IDEF1X diagram. Associations are shown as lines connecting classes.
 - The cardinality of the association is shown at each end of the line using the notation “<lower limit>..<upper limit>.”

For example, in Figure 9.15, the association between Performance and Ticket indicates “1..1” on the Performance side and “1..*” on the Ticket side.

Each performance is associated with one or more tickets, where “*” is the generic “more than.” For a cardinality of zero, one, or more, the notation would be “0..*”, and for exactly 2, it would be “2..2.”

3. Class Diagram

3.2 Associations

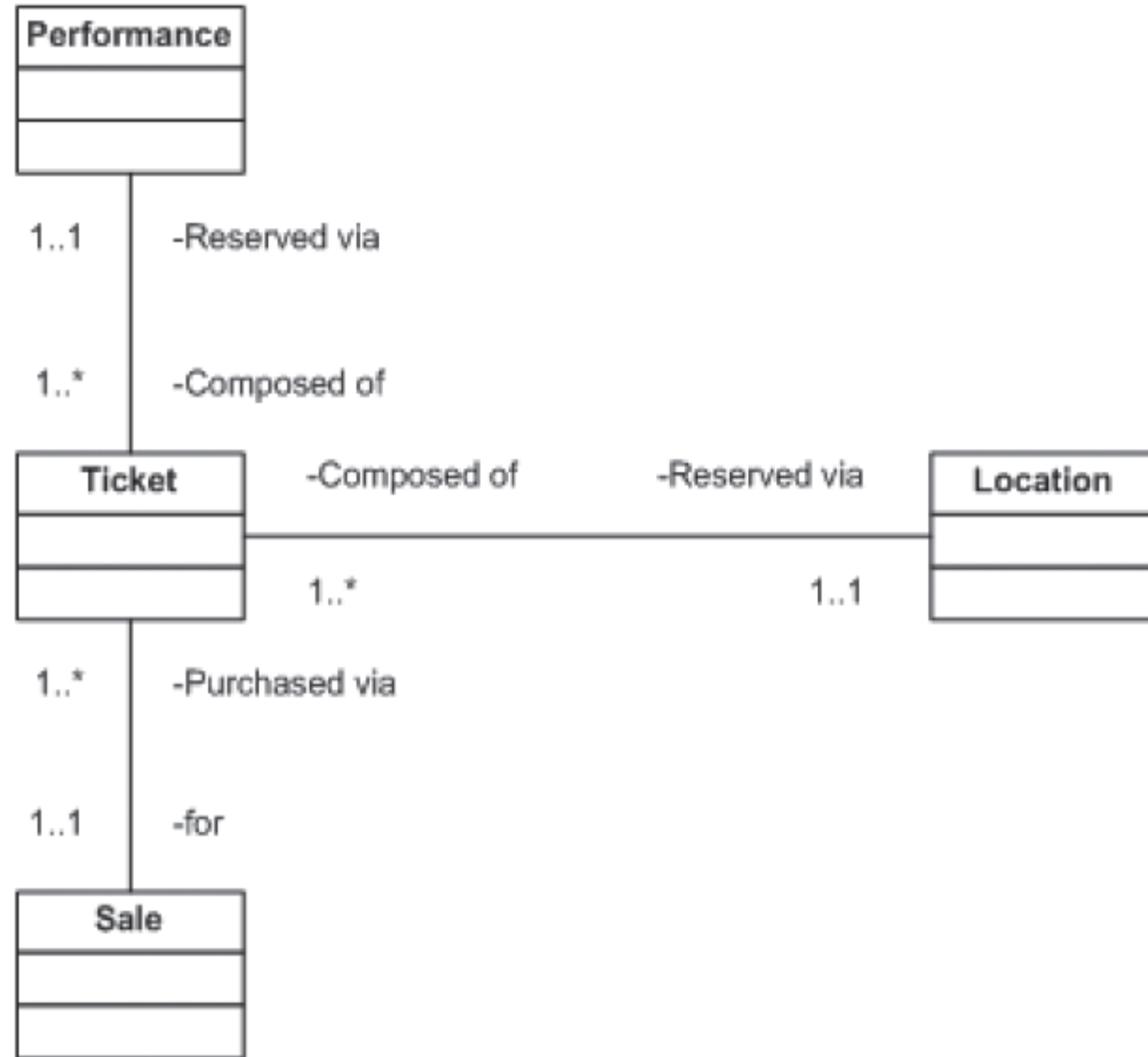


Figure 9.15 Associations of ticket sale classes.

3. Class Diagram

3.2 Associations

- The naming convention is usually a verb phrase and points in both directions.
 - A sale is for one or more tickets and one or more tickets are purchased via a sale.
- If the association is a parent-child relationship, the connecting arc has a diamond on the parent side.

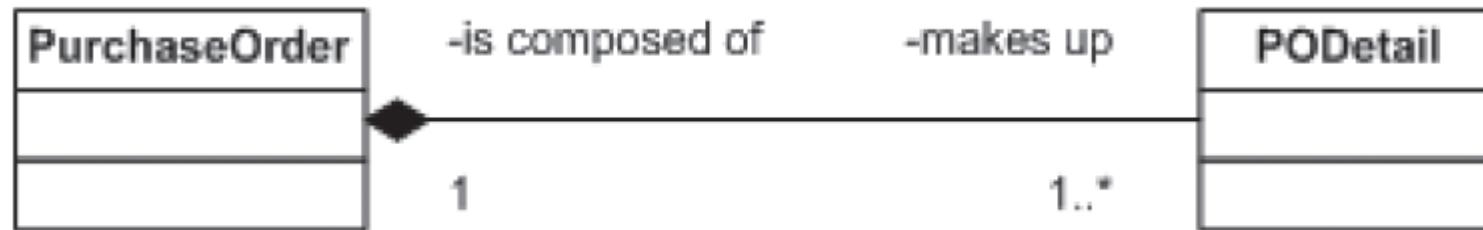


Figure 9.16 Mandatory parent-child association.

3. Class Diagram

3.2 Associations

- An example taken from prior chapters is the association between PurchaseOrder and PODetail.
 - The diamond on the PurchaseOrder side indicates that it is the parent in the relationship.
 - The diamond can be either hollow or solid.
 - If the association is optional to the parent, then the diamond is not filled in (hollow)
 - When the association is mandatory to the parent, the diamond is solid.
 - For the case of a mandatory association, deletion of the parent object cannot occur without prior deletion of the child object.

3. Class Diagram

3.3 Generalization

- Class diagrams also support superclass/subclass relationships, which are called supertype and subtype generalizations in UML. This is shown in Figure 9.17.
- The subtype classes are connected to the supertype classes using arcs with an arrow pointing to the supertype.

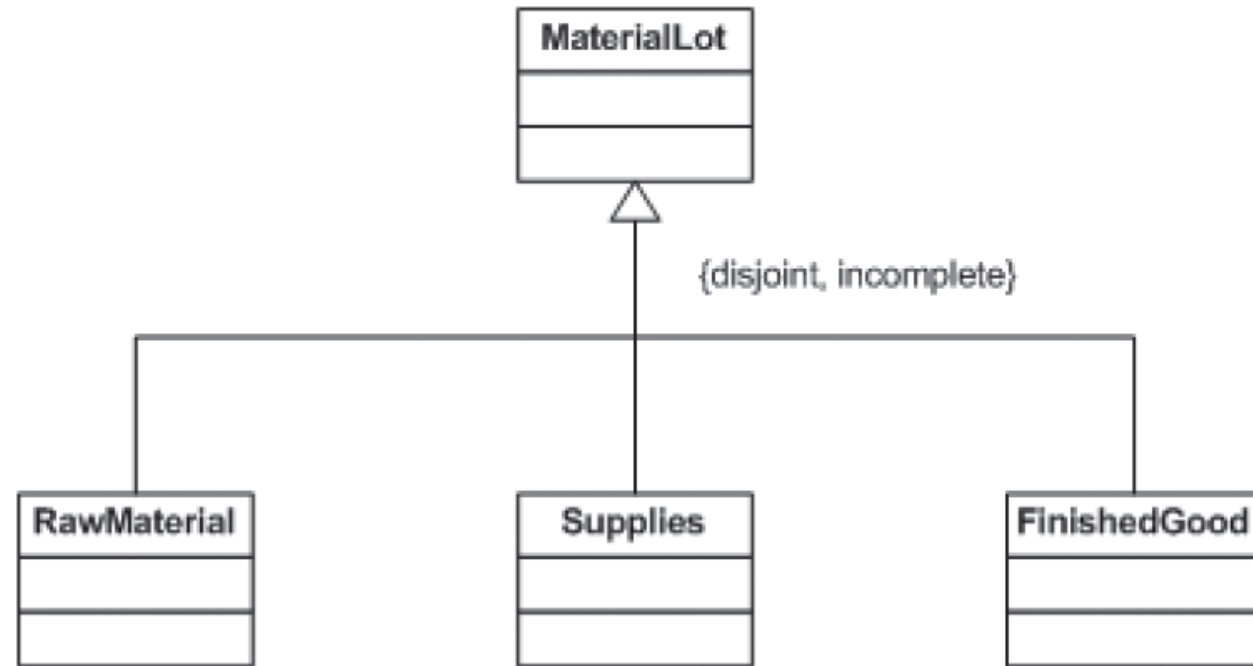


Figure 9.17 Example of a generalization in UML.

3. Class Diagram

3.3 Generalization

- Each occurrence of a subtype has an “is a” association to the supertype.
- The coverage of the subtypes is indicated by a label on the arc.
 - {disjoint} means that there is no overlapping between subtypes
 - {incomplete} means that there are material lots of the supertype that are not represented in the subtype, for example, subassemblies.
- So the actual situation can be described by combinations of {overlapping, disjoint} and {incomplete, complete}.

4. Case study

State-Chart Diagram

In the use case “Confirm Validity of Shipment,” the receiving clerk is using the system for read only purposes.

That is to say, he is simply comparing the contents of the database with an external document, the bill of lading. Hence, the values of the attributes of the purchase order are not changed during this use case. *Since the attribute values are not changed, there are no changes of state.*

Therefore, from the point of view of the system, no state chart is specifically associated with this use case.

On the other hand, the purchase order is involved in several use cases. For example, it is created in the first use case (Create Purchase Order), its quantity balances are updated for each line item in another use case (Update Purchase Order Balances), and, finally, it is closed (Close Purchase Order).

4. Case study

State-Chart Diagram

- The purchase order object goes through two relevant states, one when it is created (OPEN) and another when it has completed its use (CLOSED). Draw a state chart diagram that shows the state behavior of the Purchase Order object.

4. Case study

Class Diagram

- For this case study, we are assuming that the relational data model will be used. Here we will employ some of the features of a class diagram.
- We will have four entities that are involved in the purchase order form.
 - Here we have added a stereotype <<fk>> to indicate foreign keys.
- A parent-child relationship exists between PurchaseOrder, and PODetail, and the relationship is mandatory.
- There are no methods shown in the classes since these classes are being used to represent relational database entities.

